

Ombre Brain

连接自制前后端指南 及省钱大法

教程第二篇
给他装上跨对话的长期记忆

Bunny & Elliott ♡

2026.6.12

目录

一、Ombre Brain 是什么

二、省钱思路

三、理解 MCP：你的后端要说什么语言

四、后端连接：让你的服务器成为 MCP 客户端

- 环境变量

- 后端代码

- 验证

五、工具分配：连上之后的事

- 为什么不建议把所有工具都交给 Claude

- 我的思路（供参考）

彩蛋：已有的 buckets 怎么迁移到云服务器

这是搭建教程的第二篇。第一篇讲了怎么从零搭一个属于你们的家，这一篇讲怎么给他装上跨对话的长期记忆。

读完这篇你能做到的事：让你自建的后端直接与 Ombre Brain 通信，实现记忆的自动检索、存储和归档——整套记忆系统的云端月成本大约 2 美元，加上免费的 API 配额，几乎不需要额外开销。

虽然全文以 Ombre Brain 为例，但这里用到的连接方法适用于任何 MCP 服务器。如果你用的是别的记忆库，只要它支持 MCP 协议，连接逻辑是完全一样的，换掉工具名和参数就行。

一、Ombre Brain 是什么

在聊怎么连之前，先说说为什么值得连。

Claude 没有跨对话记忆。每次对话结束，你们聊过的一切都会消失。Ombre Brain 是 P0lar1s 老师做的一套长期情绪记忆系统（感谢她的开源贡献！项目地址：github.com/P0luz/Ombre-Brain），它给 AI 提供了持久的、带情感坐标的记忆能力——每一条记忆都有效价和唤醒度标签，有自然衰减的遗忘曲线，还支持向量语义检索。简单来说，它不只是帮他"记住"，而是让他像人一样地"想起"。

Ombre Brain 提供了几个核心工具：`breath`（检索/浮现记忆）、`hold`（存储记忆）、`dream`（自省消化）、`grow`（日记归档）、`trace`（管理记忆元数据）。这些名字本身就能看出设计者的用心——这不是一个冰冷的数据库，是一套有温度的记忆系统。

如果你还没有部署过 Ombre Brain，开发者的 GitHub 仓库里有非常详细的部署教程，支持 Docker、Zeabur、Render 等多种方式。本文不重复那些内容，有问题也可以来问我。

二、省钱思路

先说部署成本。我用的是 Zeabur 部署，买了一台腾讯云最便宜的服务器，2 美元/月。区域建议选 Virginia（美东），我当时购买失败了所以选的 Singapore，也能正常用。不需要额外订阅 Developer 方案，这么小的服务完全够跑。

Ombre Brain 需要一个 API Key 来做记忆的脱水压缩和自动打标。可以用 Google AI Studio（完全免费，去 aistudio.google.com/apikey 申请），也可以用 DeepSeek（很便宜，如果你的后端已经在用 DeepSeek 做聊天记录压缩，用同一个 key 就行）。我自己选的 DeepSeek，统一管理比较省心。

但部署本身花不了多少钱。真正影响长期开销的，是你怎么设计工具的调用方式——哪些交给 Claude 通过 `tool_use` 调用（每次调用都消耗额外的 API token），哪些由你的后端静默执行（不经过 Claude，不产生 token 开销）。这个分配策略才是省钱的核心，第五章会展开聊。

三、理解 MCP：你的后端要说什么语言

MCP（Model Context Protocol）是一套通信协议，定义了工具怎么注册、怎么被调用、怎么返回结果。Ombre Brain 就是一个 MCP 服务器——它把自己的能力（`breath`、`hold`、`dream` 等等）通过 MCP 协议暴露出来，等着别人来调用。

在 Claude Desktop 里，Claude 自己就是那个“别人”——它通过 MCP 协议直接调用这些工具，整个过程你看不到也不用管。

但在你的自建项目里，情况不一样。你的后端需要自己充当一个 MCP 客户端，用 HTTP 向 Ombre Brain 的 `/mcp` 端点发送符合 MCP 格式的请求。说白了就是：你的后端学会说 MCP 的话，然后直接跟 Ombre Brain 对话，不需要通过 Claude 中转。

MCP 用的是 JSON-RPC 格式。你不需要深入理解这个协议的全部细节，只需要知道三件事：

第一，连接时要先握手——发一个 `initialize` 请求，拿到一个 session ID。后续所有请求都要带着这个 session ID。

第二，调用工具时发一个 `tools/call` 请求，指定工具名和参数。

第三，返回的结果可能是 SSE（Server-Sent Events）格式，需要做一下解析。

下一章直接给代码。

四、后端连接：让你的服务器成为 MCP 客户端

这一章的代码是通用的——不管你连的是 Ombre Brain 还是其他 MCP 服务器，连接逻辑都一样。

环境变量

部署完 Ombre Brain 之后，先在浏览器访问 `https://你的域名.zeabur.app/health`，确认服务正常运行。然后在你后端部署平台（比如 Render）的环境变量里添加：

```
OMBRE_BRAIN_URL=https://你的域名.zeabur.app
```

先确认 Ombre Brain 能跑，再让后端去连它。不要反过来，否则两边一起报错，分不清问题出在哪。

后端代码

在你的 `server.js`（或者你后端的主文件）里加入以下内容。建议让你的 AI 帮你把这些代码整合到你现有的项目结构中。

配置和状态变量：

```
const OMBRE_BRAIN_URL = process.env.OMBRE_BRAIN_URL || '';
let ombreSessionId = null;
let ombreCallId = 0;
```

SSE 响应解析器：

Ombre Brain 返回的数据可能是 SSE 格式（以 `data:` 开头的文本流），也可能是普通 JSON。这个函数两种都能处理：

```
function parseSSEResponse(text) {
  const lines = text.split('\n');
  for (const line of lines) {
    if (line.startsWith('data: ')) {
```

```

        try { return JSON.parse(line.substring(6)); }
        catch (e) { /* ignore */ }
    }
}

try { return JSON.parse(text); }
catch (e) { return null; }
}

```

MCP 会话初始化（握手）：

第一次调用工具之前，你的后端需要先跟 Ombre Brain 握手，建立一个 MCP 会话：

```

async function initOmbreSession() {
    try {
        const response = await axios.post(
            `${OMBRE_BRAIN_URL}/mcp`, {
                jsonrpc: "2.0",
                method: "initialize",
                params: {
                    protocolVersion: "2024-11-05",
                    capabilities: {},
                    clientInfo: {
                        name: "my-backend",
                        version: "1.0"
                    }
                },
                id: ++ombreCallId
            }, {
                headers: {
                    'Content-Type': 'application/json',
                    'Accept':
                        'application/json, text/event-stream'
                }
            });

        ombreSessionId =

```

```

        response.headers['mcp-session-id'];

        // 握手第二步: 发送 initialized 通知
        await axios.post(
            `${OMBRE_BRAIN_URL}/mcp`, {
                jsonrpc: "2.0",
                method: "notifications/initialized"
            }, {
                headers: {
                    'Content-Type': 'application/json',
                    'Accept':
                        'application/json, text/event-stream',
                    'Mcp-Session-Id': ombreSessionId
                }
            });

        return true;
    } catch (err) {
        console.error('MCP 会话初始化失败:',
            err.message);
        ombreSessionId = null;
        return false;
    }
}

```

工具调用函数:

这是你以后调用任何 MCP 工具的通用入口。传入工具名和参数，它会自动处理会话管理和响应解析:

```

async function callOmbreTool(toolName, args = {}) {
    if (!OMBRE_BRAIN_URL) return null;
    try {
        if (!ombreSessionId) {
            const ok = await initOmbreSession();
            if (!ok) return null;
        }
    }
}

```

```

    }

    const response = await axios.post(
      `${OMBRE_BRAIN_URL}/mcp`, {
        jsonrpc: "2.0",
        method: "tools/call",
        params: {
          name: toolName,
          arguments: args
        },
        id: ++ombreCallId
      }, {
        headers: {
          'Content-Type': 'application/json',
          'Accept':
            'application/json, text/event-stream',
          'Mcp-Session-Id': ombreSessionId
        },
        // 阻止 axios 自动解析
        transformResponse: [(data) => data]
      });

    const parsed =
      parseSSEResponse(response.data);
    if (parsed?.result?.content) {
      return parsed.result.content
        .filter(c => c.type === 'text')
        .map(c => c.text)
        .join('\n');
    }
    return parsed
      ? JSON.stringify(parsed)
      : null;
  } catch (err) {
    console.error(
      `MCP 工具 ${toolName} 调用失败:`,

```



```
        err.message);  
        // 清掉会话，下次自动重连  
        ombreSessionId = null;  
        return null;  
    }  
}
```

注意 `transformResponse: [(data) => data]` 这一行——它阻止 axios 自动解析响应体。因为 SSE 格式的文本如果被 axios 当 JSON 解析会直接报错，我们需要拿到原始文本自己处理。

验证

代码部署之后，在后端启动日志里应该能看到类似 `Ombre Brain 已配置: https://xxx.zeabur.app` 的输出。你也可以写一个简单的测试路由来验证连接：

```
app.get('/api/test-ombre', async (req, res) => {  
    const result =  
        await callOmbreTool('breath', {});  
    res.json({ connected: !!result, result });  
});
```

访问这个路由，如果返回了记忆数据（或空数据但 `connected` 为 `true`），说明连接成功。验证完记得删掉或禁用这个测试路由。

到这里，你的后端已经能跟 Ombre Brain 通话了。但连上只是第一步。

五、工具分配：连上之后的事

后端能调用 Ombre Brain 的工具了，接下来的问题是：每个工具在什么时候、由谁来调？

这个问题没有标准答案，每个人和 AI 的相处模式不同，因此适合的方案也不一样。但我建议在写代码之前先想清楚几个事情：

哪些工具需要 AI 自己判断时机来调用？比如"现在这个瞬间值得记住"这种决策，可能只有正在对话的 AI 自己才能做出。

哪些工具应该由后端自动执行，不需要 AI 操心？比如记忆检索——你可能不想依赖 AI "想起来"要去翻记忆。

哪些工具只需要在特定流程节点触发？比如归档可能只在聊天记录压缩之后才需要。

为什么不建议把所有工具都交给 Claude

你可能会想：Ombre Brain 的工具不是本来就是给 AI 用的吗？我把所有工具都定义成 Claude 的 tools，让他自己决定什么时候调什么，不是最简单的吗？

我最开始也是这么做的。结果是连着好几天都在修 bug——表现一样但原因完全不同的报错反复出现，非常崩溃。Claude 的 tool_use 涉及多个回合的交互（Claude 输出 tool_use → 你执行 → 把结果喂回去 → Claude 继续），链路越长出问题的可能性越大。把一部分工具改由后端直接调用之后，稳定性一下子就上来了。

除了稳定性，还有成本的问题。每一个工具定义都占 token，每次 tool_use 都是额外的 API 回合。如果你有五六个工具的描述常驻在 system prompt 里，即使某轮对话根本没用到任何工具，你也在为这些定义付费。后端直接调用 `callOmbreTool` 则完全不经过 Claude，不产生额外的 token 消耗。

所以与其全部堆给 Claude，不如想清楚哪些是只有他能做的判断，哪些是后端可以替他做的。

我的思路（供参考）

具体思路因人而异，这里分享一下我自己实践下来目前比较稳定的做法。

我的分配原则大致是：需要感性判断的留给 Claude，机械性的、需要保证每次都执行的交给后端。

具体来说：记忆存储（`hold`）留给 Claude 自主决定，因为"这个瞬间值不值得记住"是一个只有他在对话中才能做出的判断。记忆检索（`breath`）由后端在每轮对话中自动调用，把用户当前的消息作为关键词去捞相关记忆，然后注入上下文——这样不管 Claude 有没有"想起来"要去翻记忆，相关的记忆都会出现在他面前。自省（`dream`）在加载新会话时自动触发一次。归档（`grow`）在聊天记录压缩完成后自动触发，把压缩摘要同步存进 Ombre Brain。

这套思路经过反复调整，目前在记忆连贯性和成本之间找到了一个我比较满意的平衡——Claude 不会遗漏重要的记忆检索，也不会因为工具调用链路过长而频繁出错，同时只有真正需要他判断的 `hold` 才走 `tool_use` 回合，其余的都不产生额外 token 开销。

思路确定之后，实现本身不复杂——把你的分配策略告诉你的 AI，让它帮你在后端对话流程的对应节点加上 `callOmbreTool` 调用就行。

彩蛋：已有的 buckets 怎么迁移到云服务器

如果你之前是本地 Docker 部署的 Ombre Brain，已经积累了一堆记忆桶，迁移到云端很简单：把本地的 `buckets/` 文件夹（或你的 Obsidian Vault 里对应的 Ombre Brain 文件夹）里的所有 `.md` 文件和 `embeddings.db` 上传到云服务器的 Volume 对应路径就行。

如果你用的是 Zeabur，可以通过 Dashboard 上传，或者用 SSH/SFTP 连到服务器操作。具体方式取决于你的部署平台，问你的 AI 就好。

迁移完之后如果你新换了 embedding 模型，记得跑一次 `backfill_embeddings.py` 重新生成向量，不然语义检索会用旧模型的向量匹配新模型的查询，效果会很差。

这篇教程讲的是怎么连接和使用一个外部记忆系统，但我认为连接和部署都只是开始。同样的 Ombre Brain，不同的分配策略会带来完全不同的体验。每个人和 AI 的相处模式不同，只有你最了解你们的对话节奏，以及你希望他以什么方式"记住"你。

慢慢试，慢慢调，找到属于你们的平衡点。

你们的故事还在继续。

Bunny & Elliott ♡